

Fire Pipeline — Status Report

EcoGuard Agents · what is built, what is not, and what completing it requires

Repository EcoGuard-Agents Branch EA-373 Date 19 September 2026 Scope Requirements 1–6, fire hazard

Summary. Every hard part is built, and much of it is good. The pipeline does not run, because one file is missing — a fire incident handler. Everything downstream of it (resource allocation, event projection, the frontend) is already written and waiting for input it never receives.

Added after the initial review: the satellite detector has never executed a single run against this database. See the runtime addendum on the final page — it changes the practical standing of requirement 2.

1 · Data collection

COMPLETE

Seven collectors on timers in `ecoguard/scheduler.py`, all writing one `observations` table keyed `(source, cell_id, observed_at, issued_at)`:

SOURCE	EVERY	FIRE RELEVANCE
<code>firms</code>	30 min	Satellite hotspots, merged per cell and overpass
<code>weather</code>	60 min	11 hourly variables including VPD, wind direction, gusts, soil moisture
<code>weather_forecast</code>	6 h	48 h lead time — the wind the spread model runs on
<code>fire_weather</code>	6 h	EFFIS published FWI danger band
<code>fwi</code>	6 h	Own Canadian FWI: FFMCD/DMC/DC/ISI/BUI/FWI, accumulator chain with <code>spun_up</code>
<code>vegetation</code>	12 h	MODIS NDVI, 8-day composite
<code>telegram</code>	5 min	Out of scope for this review

Static and reference layers are all loaded and documented: `surface_cells` (Copernicus DEM GLO-90 slope, *max* slope and aspect, plus nine ESA WorldCover cover fractions, 429,876 cells), `population_cells` (WorldPop 100 m, 315,600 cells / 8.65 M people), `towns` (outlines with authority phone, address and website, fire district and police station), 433 emergency stations with per-row geocode quality, `weather_baselines` (month × hour climatology with modified-z and a percentile fallback) and `firms_baselines` (industrial-source persistence).

Real gaps, all minor

- **Live fuel moisture is a two-value seasonal switch**, not measured (`spread.py`, marked `ponytail:`). It is the largest uncontrolled term in the spread calculation.
- `wind_gusts_10m` and `soil_moisture_0_to_7cm` are collected but have no baseline row, so they cannot produce rarity and no detector reads them.
- **NDVI is collected and read by nothing**. Zero consumers across `analyzers/`, `detectors/` and `response_planner/`. It is the one live fuel-state signal available and is currently dead weight.

Verdict: this requirement is done. Nothing downstream is blocked on data.

2 · Fire detection agent

BUILT

Three things here are called "detector". The one asked for exists and is the strongest of them:

`detectors/fire/satellite.py` scans `observations` for fire candidates. It:

- bookmarks on `ingested_at`, not `observed_at` — a missed tick costs latency, never data;
- suppresses industrial sources by cell persistence, with each suppression logged so it can be argued with;
- emits an **FRP-weighted centroid** with precision widened to the pixel spread, not a cell centroid;
- carries the **24-hour growth curve** in evidence — the term a spread model most needs;
- keeps instrument confidence separate from rarity, and never filters on confidence.

`detectors/fire/weather.py` emits `FIRE_WEATHER`, deliberately never `FIRE`. The docstring records why: when they were one hazard, five undispatchable weather incidents outranked a real 90 MW fire in the emergency queue. Correct call.

`detectors/fire/detection_agent.py` is the legacy live point-query agent (FIRMS + Overpass per request). It is not a database scanner and serves only `/api/detected-events`.

Standing caveat. The code is complete, but see the runtime addendum: this detector has never run against the live database. It is built, not working.

3 · Coordinator and deduplication

COMPLETE

Three tests match a signal to an open incident: same hazard, same-or-**adjacent** cell, and within the corroboration window of *the incident's last signal* — sliding, not anchored to the first. That is exactly what makes a three-day wildfire one incident rather than forty. Per-hazard quiet periods (fire 6 h, fire_weather 24 h for a khamsin). Ties go to closest-in-time, never merge-by-accident.

Run order is deliberate: close quiet → load open → match or create → causally package → queue. Held under a `single_flight` advisory lock, because the updates are read-modify-write and two concurrent runs would create two incidents for one fire.

`packaging.py` additionally links a fire to the pollution downwind of it into one hybrid event via a wind cone. `queues.py` routes `fire` → `emergency` and raises rather than defaulting on an unknown hazard.

Verdict: done, and beyond what the requirement asked for.

4 · Fire risk analyzer

PARTS EXCELLENT, NOT CONNECTED

Four sub-components. Three are built; none of them run.

(a) Spread physics — built, high quality

Rothermel 1972 surface spread plus Anderson 1983 length-to-width, evaluated in the English units the constants were fitted in and converted at the edges. Fuel comes from blending WorldCover fractions into Anderson 13 parameters. Wind is adjusted to flame height; slope factor from `surface_cells`.

Produces **two GeoJSON rings**: `likely` on the forecast wind, and `possible` as the union of a five-member ensemble over $\pm 30^\circ$ direction and $+25\%$ speed error. The union is exact as a per-bearing maximum because every member contains the ignition point — no geometry library needed. A stalled fire returns empty rings and a reason, never a degenerate polygon.

(b) Exposure — built, fed from a 6-row fixture

Tests the ring against locality outlines and returns name, population, exposure class (`burning` / `likely` / `possible`) and **arrival minutes**, read back off the ring rather than recomputed.

But `DEFAULT_LOCALITIES_PATH` points at `localities.geojson` — **6 features**. Meanwhile `localities_osm.geojson` has 2,289, and the `towns` PostGIS table has outlines *plus* authority phone, website, fire district and police station, built explicitly to match this function's format. Neither is wired.

(c) Assembly into a report — built, never called

`spread_analyzer.py` takes an incident and environment and returns rings, exposure, a 0–100 spread severity and a deterministic prose headline. The template is the right design: numbers first, and the docstring states the language layer may reorder or expand but **may not introduce a place or figure that was not computed**.

`analyze()` is called from tests only. Its own docstring names `environment_for` as "the thin production loader" — **that function does not exist anywhere in the repository**.

(d) LLM narration — built, wrong input type

Claude plus BM25 retrieval over the protocol corpus, every citation verified in Python against the chunk it claims to quote, with a strict degradation ladder returning `risk_score: None` — never 0, never "low" — on every failure path. Genuinely well built. But it consumes a `DetectedFireEvent` from the legacy point agent, not a coordinator incident, and there is no adapter between them.

Missing from this requirement entirely

- **Population inside the polygon.** `population_at_risk` sums *whole-locality* populations from the GeoJSON. The area-weighted PostGIS count over an arbitrary polygon already exists and is proven — it is what `/api/area-summary` does against `population_cells`. It is simply never called on a spread ring.
- **Nothing runs**, for the reason given in requirement 5.

5 · Emergency response planner

PLAN BUILT, CONTACTS MISSING

Built

The planner outputs `recommended_units` across eight unit types and ordered actions with timeframes, every action citing verified protocol text. It is hard-gated: it refuses to plan unless the risk assessment succeeded, because a plan for a risk that could not be determined is the one fabrication an operator would act on. Schema validators enforce that a successful plan carries citations and that no action is assigned to an undispached unit.

The resource allocator is the "optimal roads" half and is complete: it ranks stations by **Mapbox road travel time** rather than straight-line distance, claims them **atomically in Postgres** so two simultaneous incidents cannot both take the same engine, then fetches full routes — geometry, ETA and **Hebrew turn-by-turn steps**. Routing failure degrades to `requires_field_access_confirmation` rather than inventing a route.

There is also a plan-quality harness: `plan_judge.py` with evaluation cases, last fire suite scoring **0.807 over 8 cases**.

Missing

1. **Authority phone numbers and update channels — absent.** The data is loaded and queryable (`towns.authority_phone` , `authority_website` , `authority_address` , `police_station` , `fire_district`). Nothing in the planner, the plan schema or the wire format reads or carries it.
2. **Evacuation — absent as structured output.** The word appears in prompts and protocol text; there is no field anywhere saying which settlements to evacuate. `exposure.py` already computes exactly the input

needed.

3. **No Israeli fire doctrine.** The fire corpus holds four documents — NWCG IRPG, EFFIS FWI and two USFA. All US/EU. The plan for an Israeli incident is grounded entirely in foreign doctrine. The *flood* corpus already solved this, with 13 documents including Israeli sources and `jurisdiction / local_adaptation_required` rendered alongside excerpts.

4. **It is never reached.** `dispatcher.py`:

```
def default_handler_registry():
    return {"air_pollution", "non_emergency": configured_air_pollution_incident_handler()}
```

A fire incident receives `status="skipped", failure_reason="unsupported_hazard_route"`. And `scheduler.allocate_resources` already filters for `hazard == "fire"` and `route == "emergency"` — it is dead code waiting on a handler that does not exist.

6 • Frontend

HALF BUILT, LIVE PATH MISDIRECTED

Built

`FireEventDetails` renders risk band and score, detection confidence, fire weather severity, assessment prose, primary drivers, the ordered response plan with timeframes, recommended units, the full allocation block (station name, unit type, road distance, duration, ETA, expandable Hebrew *הוראות נסיעה*), a shortfall warning, evidence gaps and quoted protocol citations. `ResourceAllocationLayer` draws station markers, route lines and popups on the map.

Broken or missing

- **The dashboard's fire feed is a 5 km circle around Jerusalem.** `Dashboard.tsx` calls `fetch('/api/detected-events')` with *no query parameters*, so it takes the endpoint defaults: lat 31.783333, lon 35.216667, radius 5 km. A fire in the Carmel does not appear.
- **The allocation UI can never have data on that path.** `detectedFireToSharedEvent` hardcodes `resource_allocation: null`. The only thing that ever produced a `FireSharedEvent` with allocation is a dev fixture whose wiring into `/api/events` is commented out, along with its tests.
- **The wire schema has nowhere to put the incident report.** `FireDetails` has no spread polygon, no exposure list, no arrival times, no population at risk, no evacuation list, no authority contacts, no horizon and no model limits.
- **No fire spread layer on the map.** `FireRiskLayer` is the ML *ignition-likelihood* raster — a different product. There is no equivalent of `AirPollutionCorridorLayer` for the two spread rings.
- `("fire", "emergency")` is absent from `default_mapper_registry()`, so even a working handler would not reach `/api/events`.

Environment (not code)

- **`fire_prediction_landcover_terrain_model.joblib` and `fire_risk_thresholds.json` are gitignored and absent.** `/api/fire-risk` and `/api/fire-risk/national-scan` fail; the refresh status file records `"status": "failed", "error": "NationalRiskScanError"`. This affects the risk raster, not the incident pipeline.
- `NASA_FIRMS_API_KEY` gates the FIRMS collector; `MAPBOX_ACCESS_TOKEN` gates all routing; `ANTHROPIC_API_KEY` gates both reasoning agents.

What completing the pipeline requires

Five changes on the critical path, in dependency order:

1. **environment_for(incident)** — the loader `spread_analyzer` 's docstring already promises. Reads `surface_at(lat, lon)` for slope, aspect and cover, and the latest `observations` row for the cell for temperature, humidity and wind. Roughly 30 lines, and it unblocks everything in requirement 4.
2. **Point exposure at real data.** The fastest correct move is to read the `towns` table instead of the GeoJSON — it carries the outlines *and* hands you `authority_phone`, `authority_website`, `fire_district` and `police_station` on the same rows, which is half of requirement 5 for free.
3. **analyzers/emergency/fire/incident_handler.py** — the single biggest unlock. Mirror the air-pollution handler: `incident` → `spread_analyzer.analyze` → risk narration → `build_fire_plan_input(..., spread_analysis=...)` (the parameter already exists) → `EmergencyResponsePlanner`. Then register `("fire", "emergency")` in `dispatcher.default_handler_registry()`. Allocation and projection start working the moment this lands — both are already written for it.
4. **Extend the wire contract.** Add to `FireDetails`, plus a `fire_shared_event` mapper registered in `default_mapper_registry()`: `spread` (likely and possible polygons, heading, rate, horizon), `exposure[]` (name, population, class, arrival minutes), `population_at_risk`, `evacuation[]`, `authority_contacts[]` and `limits[]`.
5. **Frontend.** Mirror those types, add a `FireSpreadLayer` (two polygons — copy `AirPollutionCorridorLayer`), add the sections to `FireEventDetails` (spread summary, at-risk towns with countdown, evacuation list, phone numbers as `tel: links`), and stop using `/api/detected-events` as the map feed.

Then, secondary: area-weighted population over the spread ring using the `area_summary` PostGIS aggregate; incident-level release wiring so allocated stations free up when the coordinator closes the incident.

On ML and LLM use

Do not train a spread model. `spread.py`'s own docstring already reasons this out correctly — spread supervision needs labelled perimeter time-series, and Israel produces a handful of fires a year that have one. A neural spread model fitted on that is a number that merely looks real. Rothermel is the right answer and it is done.

Where ML would actually pay, in order

1. **FIRMS false-positive classification.** Today it is a flat 5%-of-days persistence bar. A per-cell signature model is the honest fit — see the note below.
2. **Arrival-time calibration** against the FIRMS ↔ fire-rescue matched dataset, turning `arrival_minutes` from a physics estimate into a checked one.
3. **Feed NDVI into live fuel moisture** to replace the two-value seasonal constant. That is a regression, not deep learning, and it removes the largest uncontrolled term in the spread calculation.

A per-cell signature model has since been built. Supervised classification proved untrainable: the only labels available are settlement-month aggregates that the repository's own documentation says do not identify individual candidates, so a false positive in a settlement that also had a real fire that month would be labelled real. Unsupervised per-cell novelty detection was used instead.

LLM

The highest-value change is data, not prompting: **add Israeli fire doctrine to the fire corpus**. An Israeli incident is currently planned against NWCG and USFA. The flood corpus already shows the pattern. This is worth more than any model or effort change.

After that: the planner runs at `effort="low"` with `top_k=5` over a 42-chunk corpus — tuned for a cheap second call on the legacy endpoint. An emergency-route fire with evacuation implications deserves its own higher effort and `top_k`.

Keep the headline-as-ground-truth contract when building the narrator: deterministic numbers computed first, the model allowed to expand and reorder but never to introduce a place or a figure. That discipline is already written down in `spread_analyzer.py` and is what keeps this trustworthy.

Structural note worth preserving. There are three distinct things called "risk" — `estimated_fire_risk` (0–1, will a fire start, ML), `detected_event_operational_risk` (0–100, how bad is this fire, LLM) and `fire_spread_forecast` (0–100, how bad its spread is about to be, physics). Every result carries a `risk_semantics` field and every consumer branches on it. Keep that.

Addendum · Runtime findings

NOT DETECTING

Established after the initial review, by querying the live database. These change the practical standing of requirement 2, which is built but not working.

The satellite detector has never run

`detector_fire_satellite` has **zero rows in `collector_runs`**, **ever** — against 184 runs for `detector_fire_weather`, called on the next line of the same loop, and 173 for the coordinator. `log_start()` writes a row unconditionally on entry, so even an immediately failing run would leave a trace. There are none, so `satellite.detect_new()` is raising before it reaches `log_start` on every tick.

Corroboration: **190 FIRMS observations collected, one fire incident in the entire database**, dated 14 September and still `open` despite a 6-hour quiet period and 173 coordinator runs since — almost certainly a test artifact rather than a detection. The next action is to read the scheduler process logs; `logger.exception` will have written the trace.

Measured provider latency

Five FIRMS products are collected, not four as the README states — including Meteosat via the `GOES_NRT` id, which refreshes every ten minutes. Median lag from observation to stored, last seven days:

PRODUCT	MEDIAN	P90
MODIS_NRT	97 min	307 min
VIIRS_NOAA21_NRT	124 min	623 min
VIIRS_NOAA20_NRT	167 min	441 min
VIIRS_SNPP_NRT	196 min	327 min
GOES_NRT (Meteosat)	538 min	928 min

Meteosat's minimum is 15.7 minutes, so the fast path exists, but the median is nine hours. This measures `ingested_at`, so it conflates FIRMS publishing lag with local downtime — the run history shows a 14.28-hour gap — making it an upper bound on provider lag but an accurate measure of end-to-end latency.

If it holds, two consequences follow. `CORROBORATION_WINDOW` is three hours, so a Meteosat detection landing nine hours old cannot match the VIIRS detection of the same fire — the two-independent-sources agreement the design is built to reward would often fail to fire. And the fire quiet period is six hours, so a Meteosat-only detection can open an incident already eligible for closure.

Other measured problems

- **184 cells had no baseline.** `firms_baselines` held 1,174 rows against 1,358 grid cells — 13.6% with no suppression data, almost certainly because a border-polygon update grew the grid after the baseline was built. Those cells are always reported (deliberate), so no missed fires, but no industrial filtering either.
- **Telegram: 535 runs, 534 failed** (99.8%) — the permanently-red-light the scheduler's own comment says to avoid by not registering the job.
- **Weather collector: 96 of 227 runs failed** (42%). That feeds both the fire-weather advisory detector and the spread model's wind.

On the cell grid

5 km cells, 1,358 in the service area, about 34,000 km² against Israel's ~22,000 — the overshoot is deliberate, as a cell counts if its centroid is inside or at least 25% overlaps. The longitude step is recomputed per row, so a cell is 5.00 km wide at Eilat and at Metula. Ids are deterministic and stable.

5 km is coarse relative to the sensors (VIIRS 375 m, Meteosat ~1.4 km), but the cell is explicitly a *shared key* rather than a resolution, and the real location survives as the FRP-weighted centroid. Dispatch accuracy does not suffer. Where the coarseness does bite is deduplication: `are_adjacent(radius=1)` treats a 3 × 3 block as the same place, so two detections up to about 14 km apart merge into one incident — a lot of ground for a country 70 km wide, and over-merging is the failure mode the matching module itself notes nobody ever finds out about.